

爱团部署文档

本文面向接手部署本项目的其他使用者，说明如何准备配置、构建产物、部署服务、验证运行状态，以及如何接入 CI/CD 自动化部署。

文中所有路径、域名、账号和密钥均使用占位符，不包含任何个人环境、真实服务器地址、真实密钥或固定绝对目录。部署时请按自己的服务器情况替换。

爱团部署文档

1. 文档范围
2. 系统组成
3. 部署前准备
 - 3.1 基础环境要求
 - Docker Compose 部署
 - 常规配置安装
 - 构建机建议准备
 - 3.2 仓库目录说明
4. `.config` 配置方法
 - 4.1 JWT 配置
 - 4.2 AI 助手配置
 - 4.3 邮箱配置
 - 4.4 图片上传配置
 - 4.5 地图配置
 - 4.6 `.config` 权限建议
5. `deploy/.env` 配置方法
6. 部署方式一：Docker Compose 部署
 - 6.1 Docker Compose 部署结构
 - 6.2 手动 Compose 部署流程
 - 步骤 1：准备服务器目录
 - 步骤 2：上传项目部署文件
 - 步骤 3：构建部署产物
 - 步骤 4：检查 Compose 配置
 - 步骤 5：启动服务
 - 步骤 6：查看服务状态
 - 6.3 CI/CD 镜像化 Compose 部署流程
 - 步骤 1：准备 `deploy/.env`
 - 步骤 2：检查配置
 - 步骤 3：拉取镜像并启动
 - 步骤 4：验证
 - 6.4 Docker Compose 更新流程
 - 6.5 Docker Compose 回滚流程
7. 部署方式二：常规配置安装
 - 7.1 常规部署结构
 - 7.2 安装 MySQL 8
 - 7.3 配置后端数据源
 - 7.4 构建并放置后端 JAR
 - 7.5 启动后端
 - 7.6 部署前端静态产物
 - 7.7 配置 Nginx
 - 7.8 常规部署更新流程
8. CI/CD 与自动化部署
 - 8.1 自动化目标

- 8.2 GitHub Actions 相关文件
 - 8.3 CI/CD 需要的仓库变量和密钥
 - 8.4 镜像标签策略
 - 8.5 自动化部署前置条件
- 9. 部署后验证
 - 9.1 基础健康检查
 - 9.2 页面入口检查
 - 9.3 登录和业务烟测
 - 9.4 数据库迁移检查
- 10. 常见问题排查
 - 10.1 后端启动失败
 - 10.2 `/actuator/health` 返回 502 或连接失败
 - 10.3 前端页面能打开但 API 请求失败
 - 10.4 上传图片失败
 - 10.5 邮件验证码失败
 - 10.6 AI 助手没有调用真实模型
 - 10.7 Docker 镜像拉取失败
- 11. 安全和运维建议

1. 文档范围

本文覆盖两种部署方式：

- 1. **Docker Compose 部署**：推荐方式。通过 Docker Compose 统一运行 MySQL、后端和 Nginx。适合课程验收、演示服务器、小型生产环境。
- 2. **常规配置安装**：不使用 Docker 运行后端和 Nginx，手动安装 Java、MySQL、Nginx，并使用 systemd 或等价方式托管 Spring Boot 后端。适合已有服务器运维体系或不能使用 Docker 的环境。

本文还补充：

- `.config` 配置方法；
- `deploy/.env` 配置方法；
- 前端和后端构建方法；
- 健康检查和上线验证；
- 更新、回滚和备份建议；
- GitHub Actions / CI/CD 自动化部署流程。

2. 系统组成

爱团系统采用前后端分离架构，主要交付内容如下：

组成	技术	部署形式
用户端 APP	Flutter	Android APK，可单独分发，也可放入下载目录
用户端 Web	Flutter Web	静态文件，由 Nginx 托管在 <code>/web/</code> 路径
商家端 Web	Vue 3 + Vite	静态文件，由 Nginx 托管在 <code>/merchant/</code> 路径

组成	技术	部署形式
后台端 Web	Vue 3 + Vite	静态文件，由 Nginx 托管在 <code>/admin/</code> 路径
后端服务	Spring Boot + Java 17	JAR 运行或 Docker 容器运行
数据库	MySQL 8	持久化存储业务数据
反向代理	Nginx	静态资源托管、API 反向代理、健康检查入口
配置文件	<code>.config</code> 、 <code>deploy/.env</code>	保存运行环境、数据库、密钥和外部服务配置

推荐公开访问路径：

路径	用途
<code>/</code>	项目入口页或下载说明页
<code>/web/</code>	用户端 Web
<code>/merchant/</code>	商家端 Web
<code>/admin/</code>	平台后台 Web
<code>/api/...</code>	后端 API
<code>/actuator/health</code>	后端健康检查
<code>/downloads/...</code>	APK 或其他下载文件

3. 部署前准备

3.1 基础环境要求

Docker Compose 部署

服务器建议准备：

- Linux 服务器或可运行 Docker 的环境；
- Docker Engine；
- Docker Compose Plugin；
- 可访问外网拉取基础镜像或业务镜像；
- 至少开放 HTTP / HTTPS 对外访问端口；
- 可选：域名和 HTTPS 证书。

常规配置安装

服务器建议准备：

- Java 17；
- MySQL 8；

- Nginx;
- 可选: systemd 或其他进程守护工具;
- 可选: Maven、Node.js、Flutter。如果服务器只负责运行, 不负责构建, 可以不安装这些构建工具。

构建机建议准备

如果不在服务器上构建, 构建机需要:

- Java 17 + Maven;
- Node.js + npm;
- Flutter;
- Git;
- 可选: Docker Buildx, 用于构建镜像。

3.2 仓库目录说明

项目主要目录:

```
1 | <REPO_ROOT>/
2 | └─ apps/
3 |   └─ user_app/           # Flutter 用户端
4 |   └─ merchant_web/       # 商家端 web
5 |   └─ admin_web/          # 后台端 web
6 | └─ services/backend/     # Spring Boot 后端
7 | └─ database/
8 |   └─ migrations/        # Flyway 数据库迁移
9 |   └─ seeds/              # 演示数据 seed
10 | └─ deploy/               # Docker、Nginx、部署产物目录
11 | └─ scripts/              # 构建脚本
12 | └─ docs/                 # 项目文档
```

部署时建议在服务器上规划以下逻辑目录, 实际路径由使用者自行决定:

```
1 | <APP_DIR>                 # 项目部署目录, 存放 deploy 文件、静态产物和配置
2 | <DATA_DIR>                # 数据目录, 存放 MySQL 数据和上传文件
3 | <BACKUP_DIR>              # 备份目录, 存放每次部署前的备份
4 | <CONFIG_FILE>             # 后端业务配置文件, 对应项目模板 .config.example
```

不建议把真实密钥、数据库密码、第三方 Token 写入 Git 仓库或公开文档。

4. .config 配置方法

.config 是后端业务配置文件, 用于保存 JWT、AI、邮箱、图片上传、地图等配置。项目根目录提供模板:

```
1 | .config.example
```

部署时请复制模板为实际配置文件:

```
1 | cp .config.example .config
```

也可以将 `.config` 放在服务器上的其他位置，并通过环境变量 `AITUAN_CONFIG_FILE` 指定：

```
1 | export AITUAN_CONFIG_FILE=<CONFIG_FILE>
```

后端默认会尝试读取：

```
1 | ${AITUAN_CONFIG_FILE}
2 | .config
3 | ../../.config
```

Docker Compose 部署中通常将宿主机 `.config` 挂载到容器内：

```
1 | AITUAN_CONFIG_FILE: /app/.config
```

4.1 JWT 配置

必须设置强随机 JWT secret：

```
1 | aituan.security.jwt-secret=<LONG_RANDOM_SECRET>
```

生成示例：

```
1 | openssl rand -base64 48
```

注意：

- 不要使用模板中的默认值；
- 更换 JWT secret 会让已有登录 Token 失效；
- 生产环境不要把 JWT secret 写入仓库。

4.2 AI 助手配置

AI 助手配置项：

```
1 | aituan.ai.enabled=false
2 | aituan.ai.api-url=<AI_API_BASE_URL>
3 | aituan.ai.api-key=<AI_API_KEY>
4 | aituan.ai.model=<MODEL_NAME>
5 | aituan.ai.timeout-seconds=20
6 | aituan.ai.max-tokens=800
7 | aituan.ai.temperature=0.25
```

说明：

配置	说明
<code>aituan.ai.enabled</code>	是否启用真实 AI 模型调用。为 <code>false</code> 时走本地降级回复。
<code>aituan.ai.api-url</code>	AI 服务基础地址。后端会拼接或识别 <code>/v1/chat/completions</code> 。
<code>aituan.ai.api-key</code>	AI 服务密钥。不得提交到 Git。
<code>aituan.ai.model</code>	模型名称。
<code>aituan.ai.timeout-seconds</code>	请求超时时间。
<code>aituan.ai.max-tokens</code>	单次回复最大 token 数。
<code>aituan.ai.temperature</code>	生成随机性参数。

AI 降级策略：

- `enabled=false`；
- `api-key` 为空；
- 外部模型接口超时；
- 模型返回为空；

以上情况不会阻断核心业务，后端会使用本地规则和业务 Skill 查询结果生成回复。

4.3 邮箱配置

如果需要邮件验证码，配置 SMTP：

```
1  aituan.mail.enabled=true
2  aituan.mail.debug-return-code=false
3  spring.mail.host=<SMTP_HOST>
4  spring.mail.port=<SMTP_PORT>
5  spring.mail.username=<SMTP_USERNAME>
6  spring.mail.password=<SMTP_PASSWORD_OR_AUTH_CODE>
7  spring.mail.properties.mail.smtp.ssl.enable=true
8  spring.mail.properties.mail.smtp.starttls.enable=false
9  aituan.mail.from=<MAIL_FROM>
10 aituan.mail.from-name=爱团
```

开发或演示环境可关闭真实邮件发送：

```
1  aituan.mail.enabled=false
2  aituan.mail.debug-return-code=true
```

生产或公开演示环境不建议开启 `debug-return-code=true`。

4.4 图片上传配置

本项目支持本地文件存储，也支持外部图床服务。

本地存储示例：

```
1 | aituan.upload.strategy=local
```

如果使用外部图床，请配置对应地址和 Token：

```
1 | aituan.upload.strategy=lskypro
2 | aituan.upload.lskypro.api-url=<UPLOAD_API_URL>
3 | aituan.upload.lskypro.token=<UPLOAD_TOKEN>
4 | aituan.upload.lskypro.token-url=<UPLOAD_TOKEN_URL>
5 | aituan.upload.lskypro.strategy-id=<STRATEGY_ID>
6 | aituan.upload.lskypro.album-id=<ALBUM_ID>
7 | aituan.upload.lskypro.permission=1
```

真实 Token 不得提交。

4.5 地图配置

默认可使用本地距离估算：

```
1 | aituan.map.provider=local
```

如果需要接入地图服务：

```
1 | aituan.map.provider=amap
2 | aituan.map.amap.web-api-key=<MAP_WEB_API_KEY>
3 | aituan.map.amap.geocode-url=<GEOCODE_URL>
4 | aituan.map.amap.distance-url=<DISTANCE_URL>
5 | aituan.map.amap.regeo-url=<REGEO_URL>
6 | aituan.map.amap.route-url=<ROUTE_URL>
```

地图服务不可用时，应优先保证浏览、下单、订单等核心流程可继续运行。

4.6 .config 权限建议

- .config 应只允许部署用户和后端运行用户读取；
- Docker 容器内如果后端以非 root 用户运行，宿主机 .config 需要保证容器用户可读；
- 不要在终端、录屏、PPT 或日志中输出完整 .config 内容。

5. deploy/.env 配置方法

deploy/.env 是 Docker Compose 使用的环境变量文件。模板：

```
1 | deploy/.env.example
```

部署时复制为：

```
1 | cp deploy/.env.example deploy/.env
```

常用配置：

```
1 | MYSQL_DATABASE=<DB_NAME>
2 | MYSQL_USER=<DB_USER>
3 | MYSQL_PASSWORD=<DB_PASSWORD>
4 | MYSQL_ROOT_PASSWORD=<DB_ROOT_PASSWORD>
5 | AITUAN_DATA_DIR=<DATA_DIR>
6 | AITUAN_CONFIG_HOST_FILE=<CONFIG_FILE_RELATIVE_OR_ABSOLUTE>
```

CI/CD 镜像化部署还需要：

```
1 | AITUAN_IMAGE_REGISTRY=<IMAGE_REGISTRY>/<OWNER>/<REPO>
2 | AITUAN_IMAGE_TAG=<IMAGE_TAG>
3 | AITUAN_DOWNLOADS_DIR=<DOWNLOADS_DIR>
```

变量说明：

变量	说明
<code>MYSQL_DATABASE</code>	MySQL 数据库名。
<code>MYSQL_USER</code>	后端连接数据库的业务用户。
<code>MYSQL_PASSWORD</code>	业务用户密码。
<code>MYSQL_ROOT_PASSWORD</code>	MySQL root 密码。
<code>AITUAN_DATA_DIR</code>	MySQL 数据和上传文件的数据目录。
<code>AITUAN_CONFIG_HOST_FILE</code>	宿主机 <code>.config</code> 文件路径，供后端容器挂载。
<code>AITUAN_IMAGE_REGISTRY</code>	CI/CD 推送镜像的仓库前缀。
<code>AITUAN_IMAGE_TAG</code>	当前部署镜像 tag。
<code>AITUAN_DOWNLOADS_DIR</code>	Nginx 暴露的下载文件目录。

注意：

- `deploy/.env` 不应提交 Git；
- 每次部署前建议备份；
- 如果修改数据库密码，需要同步更新 MySQL 用户密码和 Compose 配置。

6. 部署方式一：Docker Compose 部署

Docker Compose 是推荐部署方式。它可以统一管理 MySQL、后端和 Nginx，降低部署复杂度。

6.1 Docker Compose 部署结构

项目提供两个 Compose 文件：

文件	场景	特点
deploy/docker-compose.server.yml	手动构建产物部署	后端镜像在服务器本地由 JAR 构建，Web 静态产物通过目录挂载。
deploy/docker-compose.cicd.yml	CI/CD 镜像化部署	后端和 Web 镜像从镜像仓库拉取，服务器不负责 Maven/npm/Flutter 构建。

建议：

- 课程演示或手动部署：可使用 `docker-compose.server.yml`；
- 自动化部署或多人协作：推荐使用 `docker-compose.cicd.yml`。

6.2 手动 Compose 部署流程

步骤 1：准备服务器目录

在服务器创建部署目录、数据目录和备份目录。目录名由使用者自行决定，以下使用占位符：

```
1 mkdir -p <APP_DIR>
2 mkdir -p <DATA_DIR>/mysql <DATA_DIR>/uploads
3 mkdir -p <BACKUP_DIR>
```

步骤 2：上传项目部署文件

服务器 `<APP_DIR>` 至少需要：

```
1 deploy/docker-compose.server.yml
2 deploy/backend/Dockerfile
3 deploy/nginx/conf.d/default.conf
4 deploy/artifacts/backend/aituan-backend.jar
5 deploy/artifacts/landing/
6 deploy/artifacts/user-web/
7 deploy/artifacts/merchant-web/
8 deploy/artifacts/admin-web/
9 deploy/artifacts/downloads/
10 .config
11 deploy/.env
```

其中：

- `.config` 是真实业务配置；
- `deploy/.env` 是 Compose 环境变量；
- `deploy/artifacts/*` 是构建后的后端 JAR 和前端静态产物。

步骤 3：构建部署产物

在构建机执行：

```
1 mvn -B -f services/backend/pom.xml test
2 mvn -B -f services/backend/pom.xml -DskipTests package
```

将后端 JAR 复制为：

```
1 deploy/artifacts/backend/aituan-backend.jar
```

构建商家端：

```
1 npm ci --prefix apps/merchant_web
2 VITE_API_BASE_URL=<PUBLIC_ORIGIN> npm run build --prefix apps/merchant_web -- --
  base=/merchant/ --outDir <REPO_ROOT>/deploy/artifacts/merchant-web --emptyOutDir
```

构建后台端：

```
1 npm ci --prefix apps/admin_web
2 VITE_API_BASE_URL=<PUBLIC_ORIGIN> npm run build --prefix apps/admin_web -- --
  base=/admin/ --outDir <REPO_ROOT>/deploy/artifacts/admin-web --emptyOutDir
```

构建用户端 Web：

```
1 cd apps/user_app
2 flutter pub get
3 flutter analyze
4 flutter test
5 flutter build web --base-href /web/ --dart-define=API_BASE_URL=<PUBLIC_ORIGIN> --output
  <REPO_ROOT>/deploy/artifacts/user-web
```

如需同时提供 APK 下载，可构建 APK 后放入：

```
1 deploy/artifacts/downloads/
```

项目也提供服务器版构建脚本，可按目标访问地址传入：

```
1 powershell -NoProfile -ExecutionPolicy Bypass -File
  scripts/build/build_all_server_artifacts.ps1 -ServerOrigin "<PUBLIC_ORIGIN>"
```

如果不需要重新打包 APK：

```
1 powershell -NoProfile -ExecutionPolicy Bypass -File
  scripts/build/build_all_server_artifacts.ps1 -ServerOrigin "<PUBLIC_ORIGIN>" -SkipApk
```

步骤 4: 检查 Compose 配置

在服务器 `<APP_DIR>` 下执行:

```
1 | cd <APP_DIR>
2 | docker compose --env-file deploy/.env -f deploy/docker-compose.server.yml config
```

如果命令能展开完整配置且没有报错, 再启动。

步骤 5: 启动服务

```
1 | docker compose --env-file deploy/.env -f deploy/docker-compose.server.yml up -d --build
```

说明:

- `mysql` 使用 MySQL 8 基础镜像;
- `backend` 根据 `deploy/artifacts/backend/aituan-backend.jar` 构建镜像;
- `nginx` 托管静态页面并代理 `/api/` 到后端;
- MySQL 数据和上传文件放在 `<DATA_DIR>`。

步骤 6: 查看服务状态

```
1 | docker compose --env-file deploy/.env -f deploy/docker-compose.server.yml ps
```

查看后端日志:

```
1 | docker compose --env-file deploy/.env -f deploy/docker-compose.server.yml logs -f backend
```

查看 Nginx 日志:

```
1 | docker compose --env-file deploy/.env -f deploy/docker-compose.server.yml logs -f nginx
```

6.3 CI/CD 镜像化 Compose 部署流程

CI/CD 部署使用:

```
1 | deploy/docker-compose.cicd.yml
```

该方式不要求服务器本地构建后端或前端, 而是从镜像仓库拉取:

```
1 | <IMAGE_REGISTRY>/backend:<IMAGE_TAG>
2 | <IMAGE_REGISTRY>/web:<IMAGE_TAG>
```

步骤 1: 准备 `deploy/.env`

```
1 MYSQL_DATABASE=<DB_NAME>
2 MYSQL_USER=<DB_USER>
3 MYSQL_PASSWORD=<DB_PASSWORD>
4 MYSQL_ROOT_PASSWORD=<DB_ROOT_PASSWORD>
5 AITUAN_DATA_DIR=<DATA_DIR>
6 AITUAN_CONFIG_HOST_FILE=<CONFIG_FILE>
7 AITUAN_IMAGE_REGISTRY=<IMAGE_REGISTRY>/<OWNER>/<REPO>
8 AITUAN_IMAGE_TAG=<IMAGE_TAG>
9 AITUAN_DOWNLOADS_DIR=<DOWNLOADS_DIR>
```

步骤 2: 检查配置

```
1 docker compose --env-file deploy/.env -f deploy/docker-compose.cicd.yml config
```

步骤 3: 拉取镜像并启动

```
1 docker compose --env-file deploy/.env -f deploy/docker-compose.cicd.yml pull
2 docker compose --env-file deploy/.env -f deploy/docker-compose.cicd.yml up -d --remove-orphans
```

步骤 4: 验证

```
1 curl -fss <PUBLIC_ORIGIN>/actuator/health
```

预期返回:

```
1 {"status": "UP"}
```

6.4 Docker Compose 更新流程

部署更新前建议备份:

```
1 stamp=$(date +%Y%m%d-%H%M%S)
2 mkdir -p <BACKUP_DIR>/$stamp
3 cp <APP_DIR>/config <BACKUP_DIR>/$stamp/config.backup
4 cp <APP_DIR>/deploy/.env <BACKUP_DIR>/$stamp/deploy.env.backup
5 # 如有旧静态产物, 也可打包 deploy/artifacts
```

手动 Compose 更新:

```
1 cd <APP_DIR>
2 docker compose --env-file deploy/.env -f deploy/docker-compose.server.yml up -d --build
```

CI/CD 镜像化更新:

```
1 cd <APP_DIR>
2 # 修改 deploy/.env 中的 AITUAN_IMAGE_TAG
3 docker compose --env-file deploy/.env -f deploy/docker-compose.cicd.yml pull
4 docker compose --env-file deploy/.env -f deploy/docker-compose.cicd.yml up -d --remove-orphans
```

6.5 Docker Compose 回滚流程

如果是手动部署产物：

1. 从备份恢复 `deploy/artifacts`、`.config` 或 `deploy/.env`；
2. 重新执行：

```
1 docker compose --env-file deploy/.env -f deploy/docker-compose.server.yml up -d --build
```

如果是镜像化部署：

1. 将 `AITUAN_IMAGE_TAG` 改回旧 tag；
2. 执行：

```
1 docker compose --env-file deploy/.env -f deploy/docker-compose.cicd.yml pull
2 docker compose --env-file deploy/.env -f deploy/docker-compose.cicd.yml up -d --remove-orphans
```

7. 部署方式二：常规配置安装

常规配置安装不使用 Docker 运行后端和 Nginx。该方式适合已有 Java、MySQL、Nginx 运维环境的服务器。

7.1 常规部署结构

建议逻辑结构：

```
1 <APP_DIR>/
2 |─ backend/
3 |   └─ aituan-backend.jar
4 |─ web/
5 |   |─ landing/
6 |   |─ user-web/
7 |   |─ merchant-web/
8 |   |─ admin-web/
9 |   └─ downloads/
10 |─ logs/
11 |─ .config
12 |─ nginx/
13 |   └─ aituan.conf
14
15 <DATA_DIR>/
16 |─ mysql/
17 |─ uploads/
```

实际目录可自行选择，文档和脚本中统一用占位符表达。

7.2 安装 MySQL 8

创建业务库和业务用户：

```
1 CREATE DATABASE <DB_NAME>
2   DEFAULT CHARACTER SET utf8mb4
3   COLLATE utf8mb4_0900_ai_ci;
4
5 CREATE USER '<DB_USER>'@'%' IDENTIFIED BY '<DB_PASSWORD>';
6 GRANT ALL PRIVILEGES ON <DB_NAME>.* TO '<DB_USER>'@'%;
7 FLUSH PRIVILEGES;
```

安全建议：

- 不要对公网开放 MySQL 端口；
- 如果必须远程连接，应限制来源 IP；
- 使用强密码；
- 生产环境定期备份数据库。

7.3 配置后端数据源

常规安装时，可以通过环境变量指定数据库连接：

```
1 export SPRING_PROFILES_ACTIVE=dev
2 export SERVER_PORT=8080
3 export AITUAN_DATASOURCE_URL="jdbc:mysql://<DB_HOST>:<DB_PORT>/<DB_NAME>?
  useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai&allowPublicKeyRetrie
  val=true&useSSL=false"
4 export AITUAN_DATASOURCE_USERNAME=<DB_USER>
5 export AITUAN_DATASOURCE_PASSWORD=<DB_PASSWORD>
6 export AITUAN_CONFIG_FILE=<CONFIG_FILE>
7 export AITUAN_UPLOAD_ROOT=<UPLOAD_DIR>
8 export AITUAN_UPLOAD_PUBLIC_PREFIX=/api/common/files
```

其中 `<CONFIG_FILE>` 指向实际 `.config` 文件。

7.4 构建并放置后端 JAR

在构建机执行：

```
1 mvn -B -f services/backend/pom.xml test
2 mvn -B -f services/backend/pom.xml -DskipTests package
```

将生成的后端 JAR 放到服务器：

```
1 <APP_DIR>/backend/aituan-backend.jar
```

7.5 启动后端

直接启动示例：

```
1 | java -jar <APP_DIR>/backend/aituan-backend.jar
```

更推荐使用 systemd 或等价进程管理工具托管。systemd 服务文件示例：

```
1 | [Unit]
2 | Description=Aituan Backend
3 | After=network.target mysql.service
4 |
5 | [Service]
6 | Type=simple
7 | WorkingDirectory=<APP_DIR>/backend
8 | Environment=SPRING_PROFILES_ACTIVE=dev
9 | Environment=SERVER_PORT=8080
10 | Environment=AITUAN_CONFIG_FILE=<CONFIG_FILE>
11 | Environment=AITUAN_DATASOURCE_URL=jdbc:mysql://<DB_HOST>:<DB_PORT>/<DB_NAME>?
    useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai&allowPublicKeyRetri
    eval=true&useSSL=false
12 | Environment=AITUAN_DATASOURCE_USERNAME=<DB_USER>
13 | Environment=AITUAN_DATASOURCE_PASSWORD=<DB_PASSWORD>
14 | Environment=AITUAN_UPLOAD_ROOT=<UPLOAD_DIR>
15 | Environment=AITUAN_UPLOAD_PUBLIC_PREFIX=/api/common/files
16 | ExecStart=/usr/bin/java -jar <APP_DIR>/backend/aituan-backend.jar
17 | Restart=always
18 | RestartSec=5
19 |
20 | [Install]
21 | WantedBy=multi-user.target
```

注意：示例中的 `<...>` 必须替换为实际值。不要把真实密码提交到仓库。生产环境可使用 systemd 环境文件或安全密钥管理工具保存敏感变量。

启动命令示例：

```
1 | systemctl daemon-reload
2 | systemctl enable <BACKEND_SERVICE_NAME>
3 | systemctl start <BACKEND_SERVICE_NAME>
4 | systemctl status <BACKEND_SERVICE_NAME>
```

7.6 部署前端静态产物

构建用户端 Web：

```

1 cd apps/user_app
2 flutter pub get
3 flutter analyze
4 flutter test
5 flutter build web --base-href /web/ --dart-define=API_BASE_URL=<PUBLIC_ORIGIN> --output
  <APP_DIR>/web/user-web

```

构建商家端：

```

1 npm ci --prefix apps/merchant_web
2 VITE_API_BASE_URL=<PUBLIC_ORIGIN> npm run build --prefix apps/merchant_web -- --
  base=/merchant/ --outDir <APP_DIR>/web/merchant-web --emptyOutDir

```

构建后台端：

```

1 npm ci --prefix apps/admin_web
2 VITE_API_BASE_URL=<PUBLIC_ORIGIN> npm run build --prefix apps/admin_web -- --
  base=/admin/ --outDir <APP_DIR>/web/admin-web --emptyOutDir

```

根路径 landing 页可放到：

```

1 <APP_DIR>/web/landing

```

APK 下载文件可放到：

```

1 <APP_DIR>/web/downloads

```

7.7 配置 Nginx

Nginx 配置示例：

```

1 server {
2     listen 80;
3     server_name <DOMAIN_OR_IP>;
4
5     client_max_body_size 8m;
6
7     location = / {
8         root <APP_DIR>/web/landing;
9         try_files /index.html =404;
10    }
11
12    location = /web {
13        return 301 /web/;
14    }
15
16    location /web/ {
17        alias <APP_DIR>/web/user-web/;
18        try_files $uri $uri/ /web/index.html;
19    }

```

```

20
21     location = /merchant {
22         return 301 /merchant/;
23     }
24
25     location /merchant/ {
26         alias <APP_DIR>/web/merchant-web/;
27         try_files $uri $uri/ /merchant/index.html;
28     }
29
30     location = /admin {
31         return 301 /admin/;
32     }
33
34     location /admin/ {
35         alias <APP_DIR>/web/admin-web/;
36         try_files $uri $uri/ /admin/index.html;
37     }
38
39     location /downloads/ {
40         alias <APP_DIR>/web/downloads/;
41         try_files $uri =404;
42     }
43
44     location = /actuator/health {
45         proxy_pass http://127.0.0.1:8080/actuator/health;
46         proxy_http_version 1.1;
47         proxy_set_header Host $host;
48         proxy_set_header X-Real-IP $remote_addr;
49         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
50         proxy_set_header X-Forwarded-Proto $scheme;
51     }
52
53     location /api/ {
54         proxy_pass http://127.0.0.1:8080;
55         proxy_http_version 1.1;
56         proxy_set_header Host $host;
57         proxy_set_header X-Real-IP $remote_addr;
58         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
59         proxy_set_header X-Forwarded-Proto $scheme;
60         proxy_connect_timeout 30s;
61         proxy_send_timeout 120s;
62         proxy_read_timeout 120s;
63     }
64 }

```

检查并重载 Nginx:

```

1 nginx -t
2 systemctl reload nginx

```

如果启用 HTTPS, 可在此基础上添加证书配置, 或使用自动证书工具生成 HTTPS 配置。

7.8 常规部署更新流程

1. 备份旧 JAR、静态产物和配置：

```
1 stamp=$(date +%Y%m%d-%H%M%S)
2 mkdir -p <BACKUP_DIR>/${stamp}
3 cp <APP_DIR>/backend/aituan-backend.jar <BACKUP_DIR>/${stamp}/aituan-backend.jar.backup
4 cp <CONFIG_FILE> <BACKUP_DIR>/${stamp}/config.backup
5 # 可按需打包 <APP_DIR>/web
```

2. 上传新 JAR 和静态产物；

3. 重启后端：

```
1 systemctl restart <BACKEND_SERVICE_NAME>
```

4. 重载 Nginx：

```
1 nginx -t
2 systemctl reload nginx
```

5. 执行健康检查。

8. CI/CD 与自动化部署

8.1 自动化目标

推荐自动化流程：

```
1 提交 main 或手动触发 workflow
2  ↓
3 运行后端测试、前端测试、Flutter 测试
4  ↓
5 构建后端 JAR、web 静态产物
6  ↓
7 构建 backend/web Docker 镜像
8  ↓
9 推送到镜像仓库
10 ↓
11 SSH 到服务器
12 ↓
13更新 deploy/.env 中的镜像 tag
14 ↓
15docker compose pull && docker compose up -d
16 ↓
17健康检查 /actuator/health
```

8.2 GitHub Actions 相关文件

项目已有 workflow：

文件	用途
<code>.github/workflows/ci.yml</code>	PR 或手动触发，运行测试和构建校验，不部署。
<code>.github/workflows/deploy.yml</code>	main 或手动触发，构建镜像、推送镜像、可选部署服务器。
<code>.github/workflows/android-apk.yml</code>	手动触发用户端 Android APK 打包，可选上传到服务器下载目录。

8.3 CI/CD 需要的仓库变量和密钥

建议使用 GitHub Variables 保存非敏感配置：

名称	示例含义
<code>SERVER_ORIGIN</code>	公开访问地址，例如 <code>https://<DOMAIN></code> ，不包含 <code>/api</code> 。
<code>SERVER_APP_DIR</code>	服务器部署目录，占位写法为 <code><APP_DIR></code> 。
<code>AUTO_DEPLOY_PRODUCTION</code>	是否 main push 后自动部署。

建议使用 GitHub Secrets 保存敏感配置：

名称	含义
<code>SERVER_HOST</code>	服务器 SSH 地址。
<code>SERVER_PORT</code>	SSH 端口。
<code>SERVER_USER</code>	SSH 用户。
<code>SERVER_SSH_KEY</code>	部署专用 SSH 私钥。
<code>SERVER_KNOWN_HOSTS</code>	服务器 SSH host key。
<code>GHCR_READ_TOKEN</code>	如果镜像仓库为私有，服务器拉镜像需要只读 token。

注意：

- SSH 私钥不应复用个人主力密钥；
- 服务器部署用户应使用最小权限；
- 不要在 workflow 日志中输出密钥。

8.4 镜像标签策略

推荐每次构建推送两个 tag：

tag	说明
sha- <code><短提交号></code>	唯一版本，推荐部署和回滚使用。
main	main 分支最新版本，便于快速查看。

服务器 `deploy/.env` 中固定当前部署版本：

```
1 AITUAN_IMAGE_REGISTRY=<IMAGE_REGISTRY>/<OWNER>/<REPO>
2 AITUAN_IMAGE_TAG=sha-abcdef1
```

回滚时改回旧 tag 后重新：

```
1 docker compose --env-file deploy/.env -f deploy/docker-compose.cicd.yml pull
2 docker compose --env-file deploy/.env -f deploy/docker-compose.cicd.yml up -d --remove-orphans
```

8.5 自动化部署前置条件

- 1. 服务器已经准备好 `<APP_DIR>`、`deploy/.env`、`.config`、`<DATA_DIR>`；
- 2. 服务器已经能运行 `docker compose`；
- 3. 服务器能从镜像仓库拉取业务镜像；
- 4. GitHub Secrets 已配置 SSH 私钥和 `known_hosts`；
- 5. `.config` 和 `deploy/.env` 中的真实密钥没有提交到仓库；
- 6. `/actuator/health` 可以被服务器本机或 Nginx 访问。

9. 部署后验证

9.1 基础健康检查

```
1 curl -fss <PUBLIC_ORIGIN>/actuator/health
```

预期：

```
1 {"status":"UP"}
```

9.2 页面入口检查

检查项	预期
<code><PUBLIC_ORIGIN>/</code>	返回入口页或下载页

检查项	预期
<code><PUBLIC_ORIGIN>/web/</code>	用户端 Web 可打开
<code><PUBLIC_ORIGIN>/merchant/</code>	商家端登录页可打开
<code><PUBLIC_ORIGIN>/admin/</code>	后台端登录页可打开
<code><PUBLIC_ORIGIN>/api/...</code>	API 可访问，未登录接口正常返回
<code><PUBLIC_ORIGIN>/downloads/...</code>	APK 或下载文件可访问

9.3 登录和业务烟测

建议准备演示账号并完成以下烟测：

- 1. 用户端登录；
- 2. 首页和八大模块浏览；
- 3. 外卖下单和模拟支付；
- 4. 商家端查看订单并推进履约；
- 5. 用户端查看订单状态；
- 6. 用户端发布评价或提交投诉；
- 7. 商家端回复评价或客服；
- 8. 后台端处理评价 / 投诉；
- 9. AI 助手入口返回正常回复或降级回复。

9.4 数据库迁移检查

后端启动时会执行 Flyway 迁移。若启动失败，重点检查：

- 数据库连接地址、用户名、密码；
- 数据库字符集和排序规则；
- 是否已执行过不兼容的历史迁移；
- `database/migrations` 和 `database/seeds` 是否随 JAR 正确打包。

10. 常见问题排查

10.1 后端启动失败

排查：

```
1 # Docker Compose
2
3 docker compose --env-file deploy/.env -f deploy/docker-compose.server.yml logs --
  tail=120 backend
4
5 # 常规安装
6 journalctl -u <BACKEND_SERVICE_NAME> -n 120 --no-pager
```

常见原因：

- `.config` 文件不存在或容器内不可读；
- JWT secret、AI key、邮箱配置格式错误；
- 数据库连接失败；
- MySQL 未启动或账号权限不足；
- Flyway 迁移失败；
- 上传目录没有写权限。

10.2 `/actuator/health` 返回 502 或连接失败

排查：

- 后端是否启动；
- Nginx 是否代理到正确后端地址；
- Docker Compose 中 Nginx 是否能通过服务名访问后端；
- 常规部署中 Nginx 是否代理到正确端口；
- 防火墙是否拦截。

10.3 前端页面能打开但 API 请求失败

排查：

- 构建时 `API_BASE_URL` / `VITE_API_BASE_URL` 是否设置为公开访问地址；
- 是否错误地包含 `/api` 后缀；
- Nginx `/api/` 反向代理是否生效；
- 后端 CORS / 网关 / HTTPS 混合内容问题；
- 浏览器控制台网络请求地址是否正确。

10.4 上传图片失败

排查：

- `aituan.upload.strategy` 是否正确；
- 本地上传目录是否存在并可写；
- 外部图床 Token 是否有效；
- Nginx `client_max_body_size` 是否小于上传文件大小；
- `.config` 是否已在后端重启后生效。

10.5 邮件验证码失败

排查：

- `aituan.mail.enabled` 是否为 `true`；
- SMTP host、port、username、password 是否正确；
- 是否需要授权码而不是登录密码；

- SSL / STARTTLS 配置是否匹配邮件服务商要求；
- 公开演示环境是否误开 `debug-return-code=true`。

10.6 AI 助手没有调用真实模型

排查：

- `aituan.ai.enabled=true`；
- `aituan.ai.api-key` 非空；
- `aituan.ai.api-url` 可访问；
- `aituan.ai.model` 配置正确；
- 服务器能访问 AI 服务；
- 后端重启后配置已生效。

如果 AI 外部接口不可用，系统会自动降级，本身不代表核心业务故障。

10.7 Docker 镜像拉取失败

排查：

- `AITUAN_IMAGE_REGISTRY` 是否正确；
- `AITUAN_IMAGE_TAG` 是否存在；
- 镜像仓库是否公开；
- 私有镜像是否已在服务器执行 `docker login`；
- 服务器网络是否能访问镜像仓库。

11. 安全和运维建议

1. **真实密钥不入库：** `.config`、`deploy/.env`、SSH 私钥、Token 不提交 Git。
2. **部署前备份：** 至少备份 `.config`、`deploy/.env`、数据库和旧产物。
3. **最小权限：** 部署用户只应拥有部署目录、Docker 或服务重启所需权限。
4. **数据库不公网暴露：** MySQL 仅内网访问或仅本机访问。
5. **开启 HTTPS：** 公开环境建议使用 HTTPS。
6. **日志脱敏：** 不要在日志中打印密码、Token、JWT secret 和 AI key。
7. **健康检查：** 每次部署后检查 `/actuator/health`。
8. **版本可追踪：** 镜像部署使用 `sha-短提交号`，不要只依赖 `latest`。
9. **定期备份数据：** 数据库和上传目录需要定期备份。
10. **升级前先测试：** 新版本先跑 CI 和本地测试，再部署公开环境。